Introduction to Intensive Programming in C++

Lecture 8 : String algorithms

Elias TSIGARIDAS

École Polytechnique

String problems

they come with the name
Stringology algorithms

- Strings frequently appear in our kind of problems
  - Reading input
  - Writing output
  - Parsing
  - Identifiers/names
  - Data
- But sometimes strings play the key role
  - We want to find properties of some given strings
  - Is the string a palindrome?
- Strings problems are (computationally) hard, when the length is huge

various applications:
especially in (computational) biology, NLP, etc

String matching

- Given a string S of length n,
- and a string T of length m,
- find all occurrences of T in S

$n > m$ (it can happen $n >> m$)

- Note:
  - Occurrences may overlap
  - Assume strings contain characters from a constant-sized alphabet

important assumption
simplifies & accelates the algorithms
eg. using (dedicated) sorting

String matching

Example:

- $S = \text{cabcababacaba}$
- $T = \text{aba}$
- Three occurrences:
  - cabc**aba**bacaba
  - cabcab**aba**caba
  - cabcababac**aba**

Contents

Naive string matching algorithm

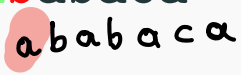
- For each substring of length m in S,
- check if that substring is equal to T.

Naive string matching algorithm

- S: bacbababaabcbab
- T: ababaca

Naive string matching algorithm

- S: bacbababaabcbab
- T: ababaca



Naive string matching algorithm

```
int string_match(const string &s, const string &t) {
    int n = s.size(),
        m = t.size();

    for (int i = 0; i + m - 1 < n; i++) {
        bool found = true;
        for (int j = 0; j < m; j++) {
            if (s[i + j] != t[j]) {
                found = false;
                break;
            }
        }
        if (found) {
            return i;
        }
    }
    return -1;
}
```

Complexity: $O(mn)$

too much if
m is big and $n >> 1$
like $O(n^2)$

What about the expected complexity?

match at the first char $1/n$

match at the first two chars $1/n^2$

⋮

Contents

Hash Function

alphabet Σ
string length n
$|\Sigma|^n$

H : strings $\longrightarrow \mathbb{Z}$
$s \longmapsto H(s)$

- A function that takes a string and outputs a number
- A good hash function has few collisions
  - i.e., If $x \neq y$, $H(x) \neq H(y)$ with high probability
- A powerful hash function is a polynomial mod a prime p
  - Consider each letter as a number (ASCII value is fine)
  - For a string s of length n, its hash value is

$$H(s) = s[1]\,A^{n-1} + s[2]\,A^{n-2} + \cdots + s[n-1]A + s[n] \pmod{p}$$

base

  - How do we find $H(s[1] \ldots s[n+1])$ from $H(s[1] \ldots s[n])$?

like evaluation of a polynomial

Q: What about perfect hashing?
Q: What about "approximations"?
Q: What about "geometric hashing"?

Hash by an example

Example

The codes of the characters of `ALLEY` are:

s_1	s_2	s_3	s_4	s_5
A	L	L	E	Y
65	76	76	69	89

If $A = 3$ and $p = 97$, the hash value of `ALLEY` is

$$(65 \cdot 3^4 + 76 \cdot 3^3 + 76 \cdot 3^2 + 69 \cdot 3^1 + 89 \cdot 3^0) \bmod 97 = 52.$$

Preprocessing (for [Karp, Rabin : 1975])

The array values can be recursively calculated as follows:

$$\begin{aligned} \mathtt{h}[0] &= \mathtt{s}[0] \\ \mathtt{h}[k] &= (\mathtt{h}[k-1]A + \mathtt{s}[k]) \bmod p \end{aligned}$$

We can construct an array P where $\mathtt{P}[k] = A^k \bmod p$:

$$\begin{aligned} \mathtt{P}[0] &= 1 \\ \mathtt{P}[k] &= (\mathtt{P}[k-1]A) \bmod p. \end{aligned}$$

} precomputations — we can do it with template metaprogramming

Hash value of a substring

Then we can calculate $\mathtt{s}[a \ldots b]$ in $O(1)$ (or almost $O(1)$)

$$(\mathtt{h}[b] - \mathtt{h}[a-1]\mathtt{P}[b-a+1]) \bmod p$$

$$H(s[a..b]) = \sum_{k=a}^{b} s[k]\, A^{k-a} \bmod p$$

$$p^a H(s[a..b]) = \sum_{k=a}^{b} s[k]\, A^k = H(s[0..b]) - H(s[0..a-1]) \Rightarrow H(s[a..b]) = \left[H(s[0..b]) - H(s[0..a-1])\right] p^{-a}$$

[Karp-Rabin: 1975] like the straightforward algo but with hash

use 10 as a base

$p = 13$

for sure it helps to precompute the powers of 10

the alphabet

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10

S

C	D	D
3	4	4

$H(s) = 3 \cdot 10^2 + 4 \cdot 10^1 + 4 \cdot 10^0 \mod 13$

$= 344 \mod 13$

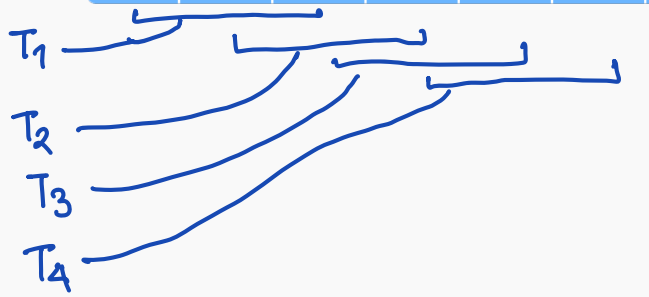
$= 6$

length $m = 3$

Consider a sliding window of size m

T

A	B	C	C	D	D	A	E	F	G



$T_1 = ABC$ $\quad H(T_1) = 1 \cdot 10^2 + 2 \cdot 10 + 3 \cdot 10^1 \mod 13 = \dots = 6$

$T_2 = BCC$ $\quad H(T_2) = 12$

$T_3 = CCD$ $\quad H(T_3) = 9$

$H(T_4) = 6$

if the hash value matches, then look at the individual characters

- spurious hit
- collision

Hash Table

- Main idea: preprocess T to speedup queries
  - Hash every substring of length k
  - k is a small constant

- For each query P, hash the first k letters of P to retrieve all the occurrences of it within T

- Don't forget to check collisions!

Hash Table

- Pros:
  - Easy to implement
  - Significant speedup in practice

- Cons:
  - Doesn't help the asymptotic efficiency
    - Can still take $\Theta(nm)$ time if hashing is terrible or data is difficult
  - A lot of memory consumption

Average complexity $O(m+n)$

What does it mean? Uniform distribution

2 Hash values (or a constant number)

TQ

How can you reduce the probability of collisions when you perform many searches?

What is the probability of collision?

TQ

Longest palindromic substring.

How to check palindromic with hashing?

$S = ABA$

$S' = ABA$

TQ : What about "geometric hashing"? LSH (Locality-Sensitive Hashing)

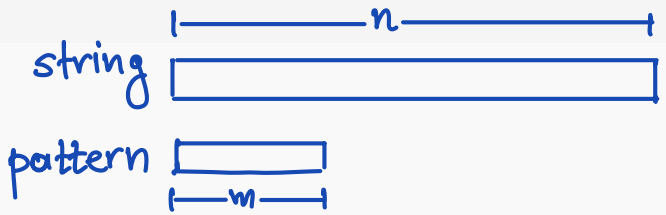
maximize collisions for NN queries

Contents

Knuth-Morris-Pratt (KMP) Matcher

~ 1970

also [Matiyasevich: 1969]



- A linear time algorithm that solves the string matching problem by preprocessing P in $\Theta(m)$ time
  - Main idea is to skip some comparisons by using the previous comparison result
- Uses an auxiliary array π that is defined as the following:
  - $\pi[i]$ is the largest integer smaller than i such that $P_1 \dots P_{\pi[i]}$ is a suffix of $P_1 \dots P_i$

π Table Example

$\pi(0) = -1$

for $P_1, \ldots, P_i$
find largest prefix that is suffix

i	1	2	3	4	5	6	7	8	9	10
P_i	a	b	a	b	a	b	a	b	c	a
$\pi[i]$	0	0	1	2	3	4	5	6	0	1

- $\pi[i]$ is the largest integer smaller than i such that $P_1 \ldots P_{\pi[i]}$ is a suffix of $P_1 \ldots P_i$
  - e.g.,, $\pi[6] = 4$ since abab is a suffix of ababab
  - e.g.,, $\pi[9] = 0$ since no prefix of length ≤ 8 ends with c

2 a b a b a b

$\pi(6) = 4$

3 a b a b a b

4 a b a b a b

Computing π

- Observation 1: if $P_1 \dots P_{\pi[i]}$ is a suffix of $P_1 \dots P_i$, then $P_1 \dots P_{\pi[i]-1}$ is a suffix of $P_1 \dots P_{i-1}$
- Observation 2: all the prefixes of P that are a suffix of $P_1 \dots P_i$ can be obtained by recursively applying π to i
  - e.g., $P_1 \dots P_{\pi[i]}$, $P_1 \dots, P_{\pi[\pi[i]]}$, $P_1 \dots, P_{\pi[\pi[\pi[i]]]}$ are all suffixes of $P_1 \dots P_i$
- A non-obvious conclusion:
  - First, let's write $\pi^{(k)}[i]$ as $\pi[\cdot]$ applied k times to i
  - e.g.,, $\pi^{(2)}[i] = \pi[\pi[i]]$
  - $\pi[i]$ is equal to $\pi^{(k)}[i-1]+1$, where k is the smallest integer that satisfies $P_{\pi^{(k)}[i-1]+1} = P_i$
    - If there is no such k, $\pi[i] = 0$
- Intuition: we look at all the prefixes of P that are suffixes of $P_1 \dots P_{i-1}$, and find the longest one whose next letter matches P_i

Implementation of π

sometimes it is called

Failure function

```
pi[0] = -1;
int k = -1;
for(int i = 1; i <= m; i++) {
  while(k >= 0 && P[k+1] != P[i])
    k = pi[k];
  pi[i] = ++k;
}
```

like applying KMP to the pattern and its prefixes

KMP Implementation

```
int string_match(const string &s, const string &t) {
int n = s.size(),
    m = t.size();

int* pi = compute_pi(t);

for (int i = 0, j = 0; i < n; ) {
    if (s[i] == t[j]) {
        i++; j++;
        if (j == m) {
            return i - m;
        }
    }
    else if (j > 0) j = pi[j];
    else i++;
}
```

KMP

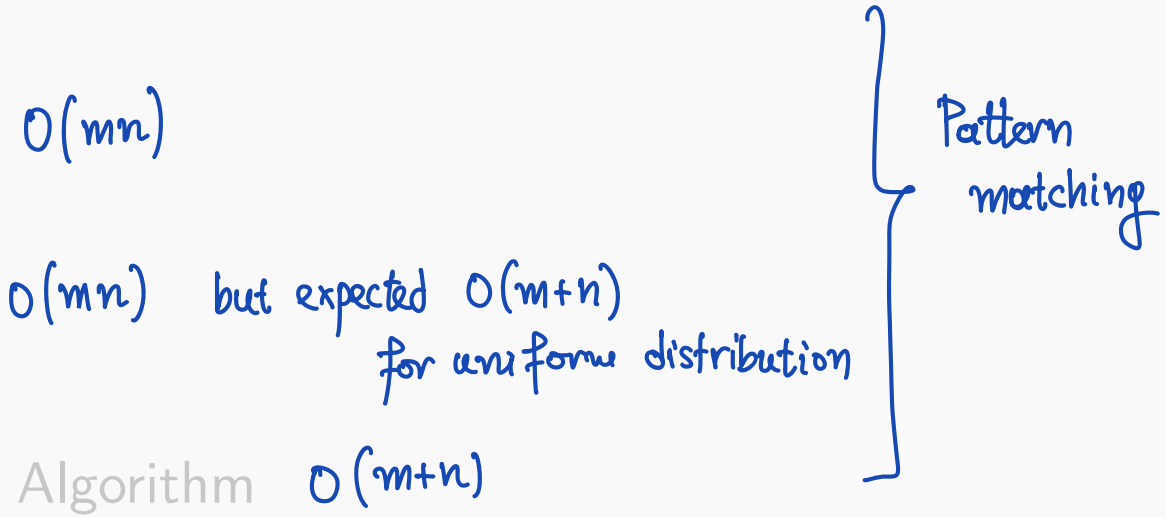
Complexity

$O(m+n)$

optimal? YES

it matches the size of the input

Contents

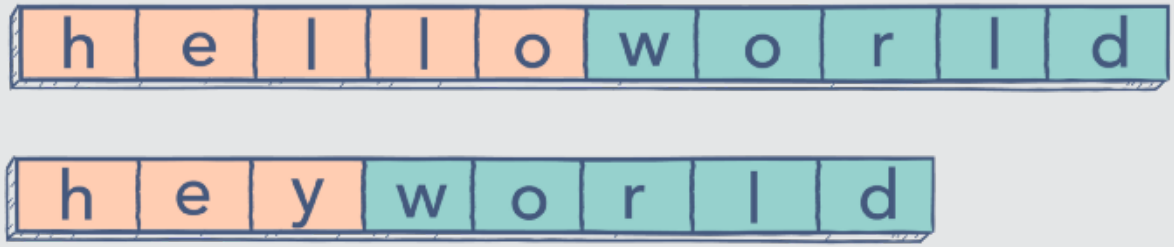


problems from DP

Longest Increasing Subsequence

Given a sequence $S = (S_1, S_2, \ldots, S_n) \in \mathbb{N}$ find the maximum number, k, of indices $1 \leq i_1 < i_2 < \cdots < i_k \leq n$, such that $S_{i_1} \leq S_{i_2} \leq \cdots \leq S_{i_k}$.

TQ: Longest Common Subsequence



TQ: Longest Palindromic Subsequence

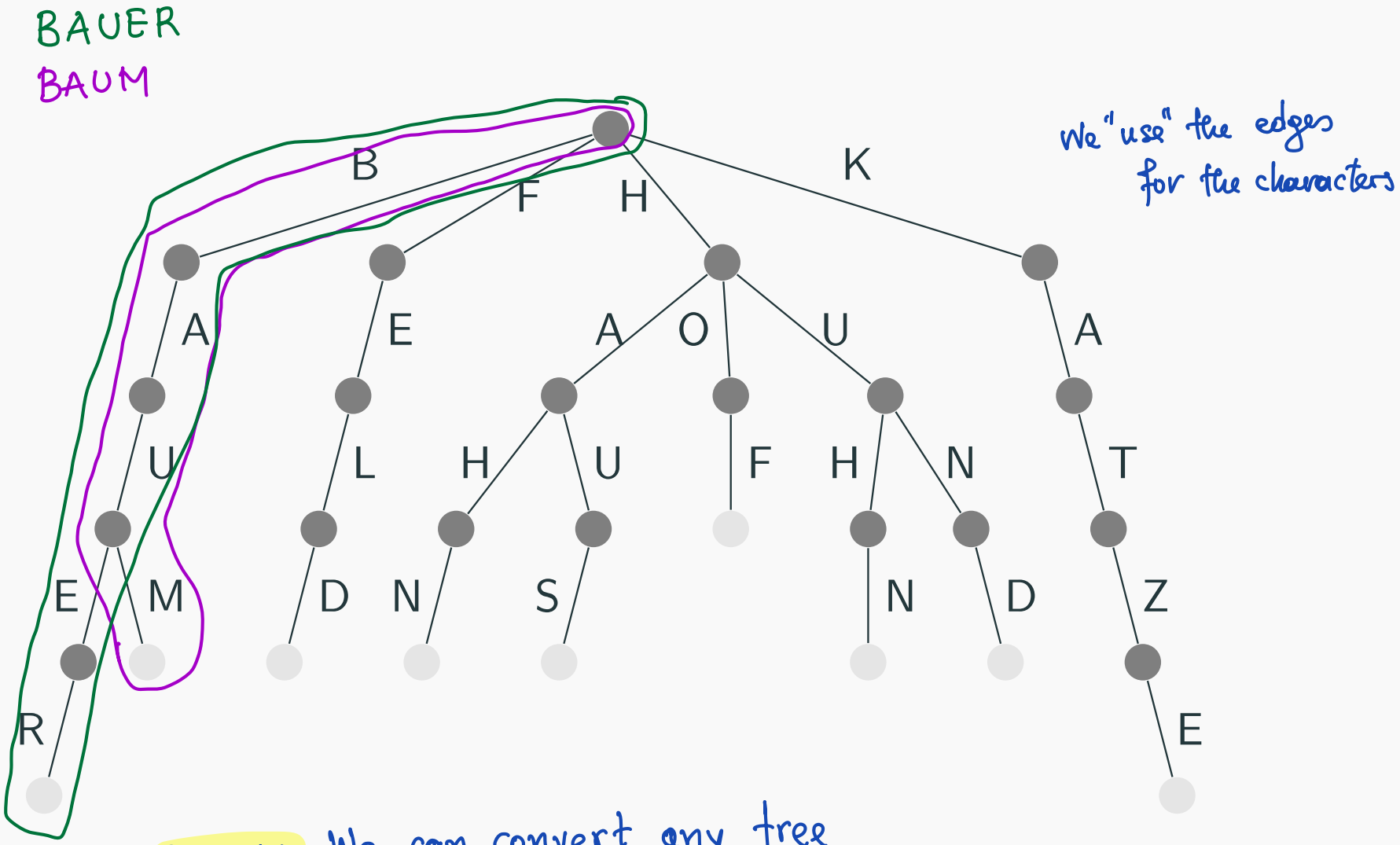
Contents

Sets of strings

- We often have sets (or maps) of strings
- Insertions and lookups usually guarantee $O(\log n)$ comparisons
- But string comparisons are actually pretty expensive.
- So that is why we use trees and tries.

Tries

from the lecture on Backtracking



RECALL: We can convert any tree to a binary tree

Tries

```
struct node {
    node* children[26];
    bool is_end;

    node() {
        memset(children, 0, sizeof(children));
        is_end = false;
    }
};

void insert(node* nd, char *s) {
    if (*s) {
        if (!nd->children[*s - 'a'])
            nd->children[*s - 'a'] = new node();

        insert(nd->children[*s - 'a'], s + 1);
    } else {
        nd->is_end = true;
    }
}
```

```
bool contains(node* nd, char *s) {
 if (*s) {
   if (!nd->children[*s - 'a'])
     return false;
   return contains(nd->children[*s - 'a'],
 } else {
   return nd->is_end;
 }
}
```

What is this?

*s 'a'

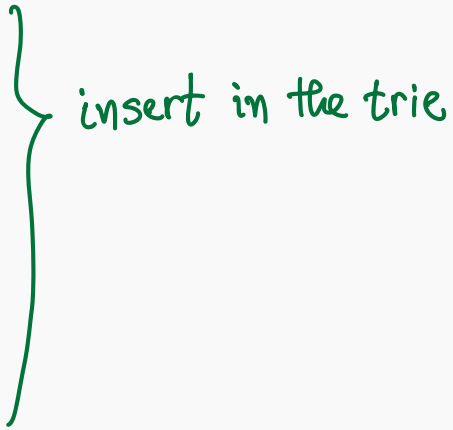
'a' - 'a' = 0

'b' - 'a' = 1

etc

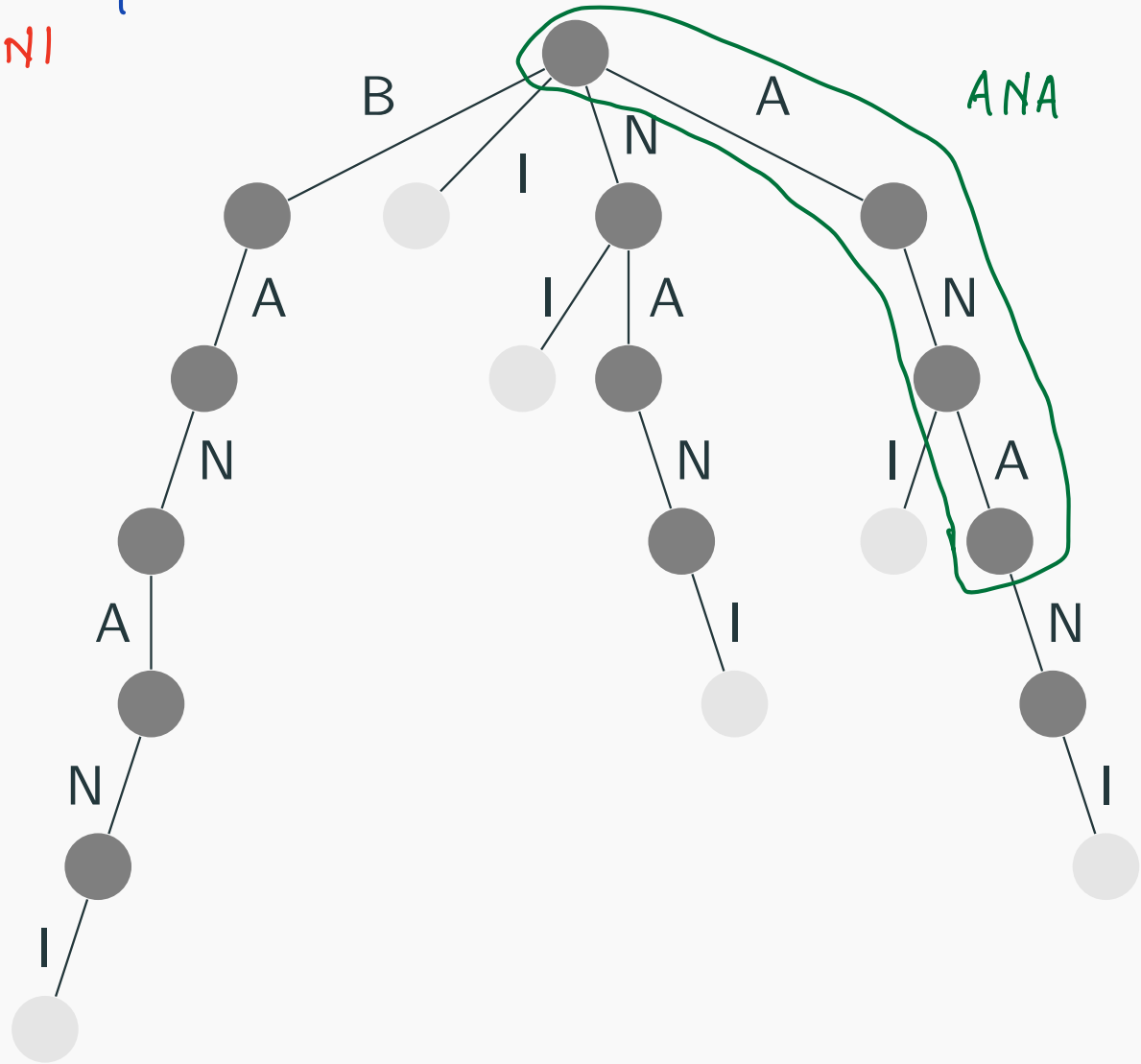
Suffix tries

- Consider a string S of length n
- Insert all suffixes of S into a trie
- $S =$ `banani`
  - `insert(trie, "banani");`
  - `insert(trie, "anani");`
  - `insert(trie, "nani");`
  - `insert(trie, "ani");`
  - `insert(trie, "ni");`
  - `insert(trie, "i");`



Suffix tries

all suffixes of BANANI

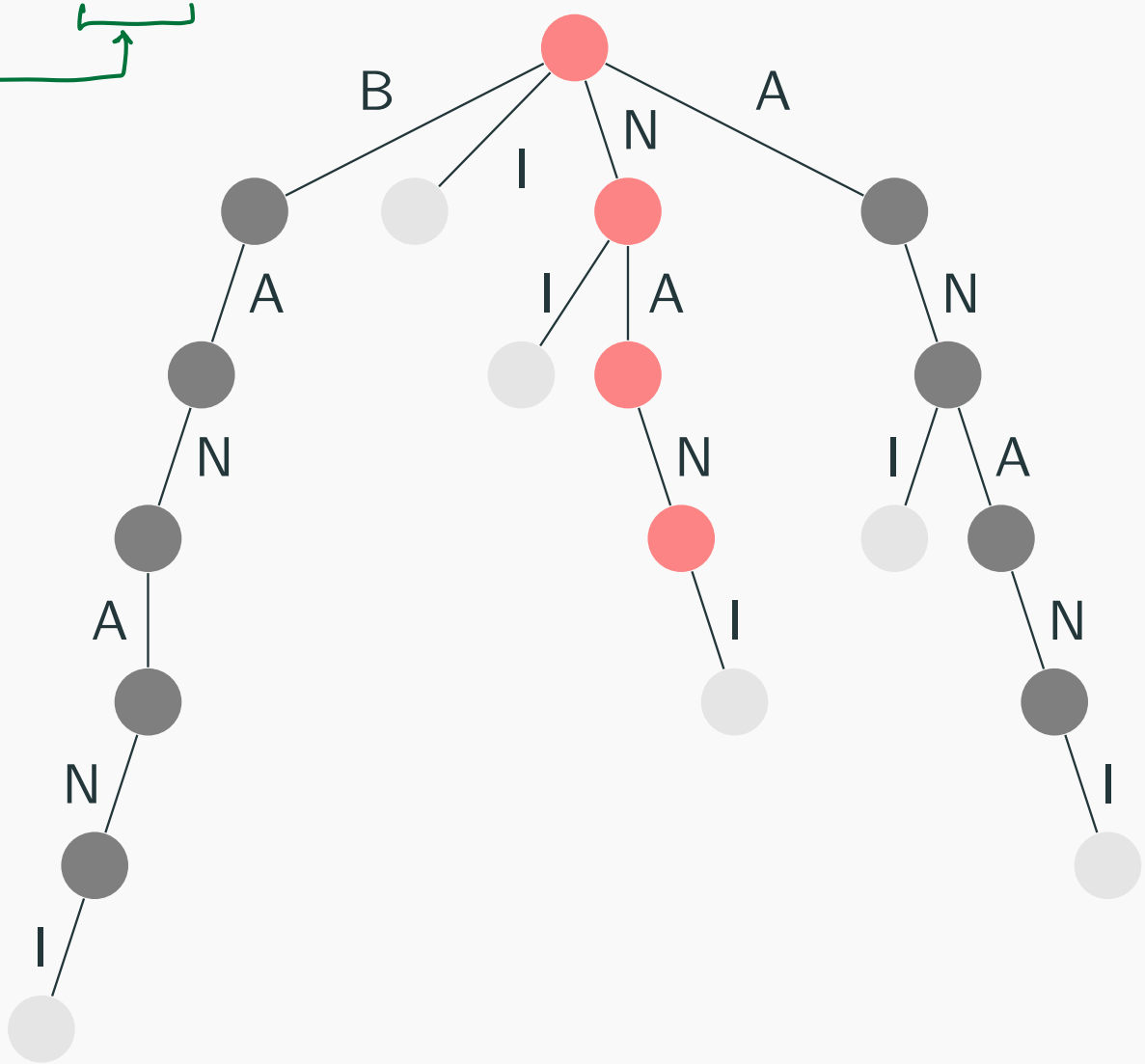


Suffix tries

- We can do string matching using suffix tries.
  - If a string T is a substring in S, then it has to start at some suffix of S
  - So look for T in the suffix trie of S (ignoring whether the last node is an end node or not)
  - This is $O(m)$ m = length of the pattern

Suffix tries

find NANI in BANANI



Suffix tries

- String matching is fast if we have the suffix trie for S
- Time complexity of suffix trie construction?
  - There are n suffixes, and it takes $O(n)$ to insert each of them
  - So $O(n^2)$, which is pretty slow
- Can we do better?
  - There can be up to n^2 nodes in the graph, so this is optimal...

Suffix arrays :: string matching

- Consider all the suffixes of S:

```
banani
anani
nani
ani
ni
i
```

- Sort all the suffixes of S:

```
anani
ani
banani
i
nani
ni
```

Find nan

. First letter n
  Binary searches to find the range of suffixes starting from n

. Second letter a
  Binary searches to find the range of suffices having second letter a

etc

Complexity $O(m \cdot \lg n)$

m: length of the query string

$\lg n$: binary search

Suffix arrays :: construction :: space

- How do we construct a suffix array for a string?
    - A simple `sort(suffixes)` is $O(n^2 \log(n))$, because comparing two suffixes is $O(n)$
- Big space requirements:
  (As with suffix tries)
  there are $O(n^2)$ characters in all suffixes

Solve the space problem, by keeping the indices

```
1: anani
3: ani
0: banani
5: i
2: nani
4: ni
```

Suffix arrays :: construction

What about the construction?

- Roughly speaking
  - sort all suffixes by only looking at the first letter
  - sort all suffixes by only looking at the first 2 letters
  - sort all suffixes by only looking at the first 4 letters
  - sort all suffixes by only looking at the first 8 letters
  - ...
  - sort all suffixes by only looking at the first 2^i letters
  - ...

2^0
2^1
2^2
2^3
2^i
$\vdots$
$2^k = n^2 \Rightarrow k = O(\lg n)$

- If we use an $O(n \log n)$ sorting algorithm, this is $O(n \log^2 n)$
- BUT, we can use an $O(n)$ sorting algorithm, because all sorted values are between 0 and n.
  So the total complexity is $O(n \log n)$.

Why?

Suffix arrays ::LCP

- Find the longest common prefix (LCP) of two suffixes of S

```
1: anani
3: ani
0: banani
5: i
2: nani
4: ni
```

- `lcp(1,3) = 2`
- `lcp(2,1) = 0`

- Time complexity $O(\log n)$
  using intermediate results from the suffix array construction

Suffix arrays

```
int lcp(int x, int y) {
    int res = 0;
    if (x == y) return n - x;
    for (int k = P.size() - 1; k >= 0 && x < n && y < n; k--) {
        if (P[k][x] == P[k][y]) {
            x += 1 << k;
            y += 1 << k;
            res += 1 << k;
        }
    }
    return res;
}
```